

How the complete system works

Feature logic and presentation script

Expanded edition | All four portals

Read-aloud scripts, detailed processing rules, formulas, status transitions, validation rules, exception paths, report calculations, and a complete active API route index.

How to use this document

Use the quoted presentation script to explain each feature aloud. Follow it with the numbered logic and example when answering questions. The contents and PDF bookmarks locate individual features.

Scope and evidence

This edition expands the earlier overview into individual workflows for Admin, Warehouse Staff, Driver, and Customer. It covers the main navigation and active routed domains found in the local source. It also explicitly records visible but unconfirmed integrations.

All numerical examples are illustrative. Rules are based on inspected source code, not a claim that every deployed workflow was exercised. No application code, configuration, or database records were changed.

Three distinctions to remember

Separate concepts	Why it matters
Physical / reserved / sellable	Goods can remain physically present while unavailable for another order.
Request / PO / trip / stop	Approval, preparation, dispatch and delivery are different events.
Full goods / empty containers / deposits	Stock quantities, collected returns and financial credit have separate records.

Feature index 1

Section / feature	Page
01 Portals and responsibility	6
02 Login, registration, and account security	7
03 Registration and email verification	8
04 Sessions, two-factor login, and password recovery	9
05 Warehouses, products, and client records	10
06 Warehouse assignment, capacity, and deactivation	11
07 Product registration, edits, and archiving	12
08 Staff, clients, privileges, and discount policies	13
09 Inventory status and Needs Restocking	14
10 Stock-in and the Overstocked rule	15
11 Stock quantities and conversion formulas	16
12 Stock-in validation, bulk intake, and retries	17
13 Batch correction and manual inventory adjustment	18
14 Batches, transactions, and expired stock	19
15 Expiry-health levels and disposal calculations	20
16 Inventory alerts, transactions, and audit interpretation	21
17 Catalog, cart selection, and Buy Again	22
18 Delivery address, map pin, and service area	23
19 Customer ordering, prices, and discounts	24
20 Online mixed-case composition and allocation	25
21 Purchase requests and approval	26
22 Request expiration, duplicate submission, and document identity	27
23 Purchase-order status and cancellation	28
24 Order status transition reference	29

Feature index 2

Section / feature	Page
25 Reservations, allocation, and mixed cases	30
26 Driver profiles, service areas, and vehicle pairing	31
27 Drivers, vehicles, and trip planning	32
28 Upcoming deliveries, route grouping, and trip changes	33
29 Trip start, delivery proof, and closure	34
30 Scheduled day, vehicle state, and explicit trip completion	35
31 Failed delivery, same-day retry, and rescheduling	36
32 POD camera, uploads, and private evidence	37
33 Live tracking, navigation, and ETA	38
34 Location freshness, retry behavior, and trip history	39
35 Replacement requests and decisions	40
36 Replacement scheduling, line progress, and closure	41
37 Returnable bottles, cases, and deposits	42
38 Deposit credit on a purchase order	43
39 Collected empties, shortages, and settlement states	44
40 Warehouse empty cases and explicit returns	45
41 Retail point of sale	46
42 Retail quote, receipt, cancellation, and units	47
43 Dashboards and reports	48
44 Warehouse and driver dashboard calculations	49
45 Report loading, date filters, export, and recognition	50
46 Purchase Requests and Purchase Orders reports	51
47 Transaction Records and Retail Sales reports	52
48 Logistics and Replacement Records reports	53

Feature index 3

Section / feature	Page
49 Top Clients report and customer ranking	54
50 Warehouse capacity and inventory reports	55
51 Feedback analytics and satisfaction calculations	56
52 Feedback, notifications, settings, and sync	57
53 In-app notifications, read state, and push devices	58
54 Profile changes and security settings	59
55 Synchronization, search, filtering, and data freshness	60
56 One complete demonstration scenario	61
57 Quick answers for a presentation or defense	62
58 Status families that must not be confused	63
59 Verified implementation limits and discrepancies	64
60 Active endpoint index 1	65
61 Active endpoint index 2	66
62 Active endpoint index 3	67
63 Active endpoint index 4	68
64 Active endpoint index 5	69

Where each feature belongs

Portal / area	Coverage in this edition
Admin	Dashboard; requests and PO monitoring; fleet/drivers/trips; live tracking; replacement decisions; inventory/stock monitoring; warehouse management; feedback; every main report tab; clients/discounts; users/roles; settings and notifications.
Warehouse Staff	Dashboard; retail; request approval/rejection; PO processing/rescheduling; route/trip planning and edits; replacement processing/scheduling; tracking; product/inventory/batch/expiry operations; transactions; empties; warehouse scope; settings.
Driver	Home/workload; assigned trips/details; load confirmation; start/route/navigation; arrival/POD; failures/retries/reschedules; collected empties; explicit completion; history; profile/security; notifications and location freshness.
Customer	Registration/login/security; catalog/cart/selection/mixed-case builder; checkout/address/service area; PR and PO detail; cancellation; Buy Again; tracking; replacement claims/cancel; feedback; empties/deposit credit; profile/settings/notifications.
Shared	Authentication, authorization, uploads/private evidence, pagination/filters/search, synchronization, duplicate-request protection, calculations, status families and implementation limits.

Read related sections together

The short feature scripts and the detailed subfeature pages are complementary. For example, restocking is covered by Inventory status, Stock-in/Overstock, quantity formulas, expiry health, corrections, and alerts. Reports have separate pages so their different date and calculation rules are not collapsed into one generic formula.

Portals and responsibility

Presentation script

"Our system connects four roles. Customers submit requests and follow their orders. Warehouse staff handle stock, approve requests, prepare orders, sell at the counter, and arrange delivery. Drivers execute assigned trips and record delivery results. Administrators monitor operations and manage accounts, clients, and replacement decisions."

How the logic works

1. The role stored in the authenticated session determines which portal and actions are available.
2. Warehouse operations use warehouse assignments so staff work with the facilities assigned to them.
3. Customer order endpoints restrict records to the signed-in customer. Driver trip endpoints select that driver's assignments.
4. The same business records connect the portals: a request becomes a purchase order, then a trip stop, then a delivery result.

Example to explain

When warehouse staff approve a request, the customer sees the resulting order state. When its trip starts, the delivery status changes for the relevant portals.

Important implementation detail

Admin inventory access is monitoring-only in the stock-write endpoints. Warehouse staff perform the stock changes; a higher-level role does not automatically mean every operational button is available.

Code basis: backend/core/models.py; auth_guards.py; views_orders.py; views_driver.py; views_customer.py; src/components/portals/admin/AdminPortal.tsx; warehouse/WarehousePortal.tsx

Login, registration, and account security

Presentation script

"The system checks the account before opening a portal. If two-factor authentication is enabled, the password or Google sign-in is only the first step. The user must also verify the login code before receiving an authenticated session."

How the logic works

1. Login handlers validate the account and issue a staff or customer session. Google login verifies the supplied Google credential through the project's verification helper.
2. Two-factor-enabled staff and customer accounts enter an OTP challenge. A pending challenge is not accepted as a normal signed-in session.
3. Registration and email-change flows use email verification. Password reset uses an OTP verification flow before allowing a new password.
4. Administrators create staff accounts and control account privileges. Account inputs include email, password, phone, and name validation.
5. Profile/security updates support two-factor settings, login alerts, and session timeout. Staff timeout updates reject values below five minutes.

Example to explain

A warehouse employee with two-factor enabled enters the correct password, receives the login challenge, verifies the OTP, and then enters the warehouse portal.

Important implementation detail

Email verification, password-reset proof, and login sessions serve different purposes. Possessing one of the first two does not grant normal portal access.

Code basis: backend/core/views_auth.py: auth_login, auth_login_verify_otp, auth_register, auth_password_reset_*; auth_guards.py: _payload; views_users.py: users_collection, user_detail

Registration and email verification

Presentation script

"A customer verifies their email before registration. The system validates the submitted identity and password, creates the customer account, and signs the customer in. This registration flow does not wait for a separate administrator approval."

How the logic works

1. Inputs include name parts or a full name, email, password, and the email-verification token. Address fields may also be supplied.
2. Names containing numbers are rejected. Email is trimmed and lowercased; duplicate customer emails are rejected. The email-verification token must match the submitted email and customer account type.
3. The password policy requires at least eight characters, uppercase, lowercase, a number, and a special character, with no spaces. The saved password is hashed.
4. If address data is supplied, the backend checks the permitted city/province and coordinates through the shared geography helper.
5. Verification request/confirm handlers issue and validate OTP-related proofs. The configured OTP-expiry constant is two minutes; a verification proof is distinct from the code and from a login session.
6. On successful creation, the login response returns the customer session. Staff account creation is a separate administrator-controlled workflow.

Example to explain

An already-registered email is rejected rather than creating another customer. An expired verification attempt must be renewed; it does not bypass registration checks.

Code basis: backend/core/views_auth.py: auth_register, auth_email_verification_*, api_constants.py; views_users.py: users_collection

Sessions, two-factor login, and password recovery

Presentation script

"A successful login produces a session for the correct account and portal. Two-factor login, password reset, and email verification use separate proofs. The system also limits repeated authentication attempts and expires applicable idle web sessions."

How the logic works

1. Password login and supported Google-login paths enter a second-factor challenge when the account has two-factor enabled. Challenge verification must succeed before normal session issuance.
2. Portal-specific HttpOnly cookies allow staff portals and customer sessions to coexist. The authentication guard accepts session types staff/customer with a user ID, not a pending proof.
3. For a web login without Keep me logged in, the session has an idle limit. Default idle time is 30 minutes; staff-configured values have a five-minute minimum. Native clients are treated differently by the client-header rule.
4. Authenticated activity updates the session activity record. Logout revokes the session; account password, email, role, or two-factor changes invalidate previous session signatures.
5. Password recovery is request OTP -> verify OTP -> reset password. The new password must pass the strength rule, and consumed reset proofs cannot be reused while protected by their record.
6. Configured policies use a 15-minute window: login/OTP failures have account limit 5 and IP limit 20; OTP sends use account limit 3 and IP limit 10. Progressive delays and temporary lockouts return HTTP 429 with Retry-After.

Example to explain

A browser may still display a cached page after a session expires, but the next protected API request must pass the session check again.

Important implementation detail

These are implementation policies, not a claim that every client has identical session duration or that authentication was penetration-tested.

Code basis: backend/core/auth.py; auth_guards.py; auth_throttling.py; views_auth.py: auth_login_verify_otp, auth_activity, auth_logout, auth_password_reset_*

Warehouses, products, and client records

Presentation script

"Warehouse and product records provide the data used by later transactions. The product defines what is sold, while inventory records how much of that product belongs to each warehouse. Client records also hold the discount policy that checkout evaluates."

How the logic works

1. A product carries its identity, packaging or unit, size, quantity per unit, prices, and applicable deposit configuration.
2. Product creation requires a warehouse. The system checks the product identity and rejects an existing combination of the same name, size, and category.
3. Quantity per unit must be positive, and the product size and quantity must support the weight calculation used for vehicle loading.
4. Inventory is associated with a warehouse and product. Stock increases and capacity changes are checked against the warehouse capacity rule.
5. Administrators manage client account details and discount settings. Customer self-service updates remain subject to ownership and field validation.

Example to explain

A 24-bottle glass case and a differently sized package cannot be treated as identical quantities. Packaging determines bottle conversion, deposits, and load calculations.

Important implementation detail

A product listing is not itself a stock movement. The inventory and transaction records provide the quantity history.

Code basis: backend/core/views_products.py: products_collection, product_detail; views_warehouses.py; views_users.py: customer_detail; models.py: Product, Inventory, Warehouse

Warehouse assignment, capacity, and deactivation

Presentation script

"An administrator registers the warehouse and assigns its staff member. Warehouse capacity is a hard case-count limit used during stock increases, not just a dashboard percentage. Deactivating a warehouse changes its active flag rather than deleting all its history."

How the logic works

1. Create requires name, code, address, city, province, ZIP code, and positive capacity. Address validation checks the supported location fields.
2. The stored manager ID represents the assigned warehouse staff member. When staffIds is submitted, the resolver uses its first non-empty value if no managerId is supplied.
3. The assignment check rejects a staff member already assigned to another warehouse. Warehouse staff list requests are restricted using manager_id assignments.
4. Used capacity = sum of Inventory.quantity across the warehouse. Projected use = current case quantity + positive incoming cases. The operation is rejected when projected use exceeds capacity.
5. Reducing configured capacity below current stored case quantity is also rejected. The hard-capacity helper does not subtract reservations and does not convert loose bottles into extra cases.
6. DELETE on the warehouse detail handler sets is_active to false and generates a staff notification; the warehouse row remains.

Example to explain

A warehouse with capacity 500 and 470 recorded cases can receive 30 additional cases. Receiving 31 is rejected. Reserving 50 of those cases does not free warehouse storage.

Important implementation detail

The scope of a list endpoint should not be assumed to prove every detail endpoint has the same guard. This document describes the checks observed in each workflow.

Code basis: backend/core/views_warehouses.py; warehouse_access.py: _get_allowed_warehouse_ids_for_staff, _warehouse_capacity_error

Product registration, edits, and archiving

Presentation script

"Products define identity, packaging, prices, and weight. Warehouse staff register and maintain them. When a product is no longer used, the archive operation preserves its historical references and is blocked until its stock and reservations are cleared."

How the logic works

1. Product creation requires SKU/name and a warehouse. A duplicate name-size-category identity is rejected. Positive quantity-per-unit and valid size data are needed for packaging and weight.
2. Initial available quantity must be a non-negative integer. Creating initial stock uses warehouse capacity checks. Applicable bottle/case deposit values are validated.
3. Product edits update the supplied fields and recheck duplicate identity, warehouse permission, sizes, weight inputs, and deposit amounts where applicable.
4. Archive checks all product inventory for cases, loose bottles, reserved cases, and active base-unit reservations. Any remaining stock or reservation blocks archival.
5. For an allowed archive, missing historical order-item name/SKU/unit snapshots are filled from the product, then `is_active` is set false. Protected transaction and packaging history remains.
6. Catalog reads use active/real-product filters. An archived product should not be presented as a currently available item merely because an old order references it.

Example to explain

Zero case stock is insufficient for archival when three loose bottles remain. Those bottles still constitute physical stock.

Code basis: `backend/core/views_products.py`: `products_collection`, `product_detail`; `product_weights.py`; `models.py`: `Product`, `OrderItem`

Staff, clients, privileges, and discount policies

Presentation script

"The Users and Clients areas manage identities and commercial settings. Creating a staff account does not automatically assign a warehouse, a vehicle, or a trip. Those operational relationships have separate rules."

How the logic works

1. Administrator-created staff accounts require name, email, password, and a valid role. Email verification and input checks precede creation.
2. Privilege changes are restricted to administrators. Users changing their own sensitive fields must satisfy the relevant OTP/email-verification flow.
3. Client account management is administrator-controlled, while a customer can update permitted fields on their own record. The server uses the authenticated account to enforce ownership.
4. Client discounts support no discount, configured 5/10/15/20/25 percent presets, and a custom percentage. Cancelled or removed policies do not qualify at checkout.
5. A configured policy still requires at least 50 eligible cases in the order. The saved order includes the applied discount metadata, so later policy changes do not mean a historical total should be recomputed.
6. Driver service-area assignment is a separate administrator action. Operational driver license/contact and fleet assignment updates are handled by warehouse operations.

Example to explain

A client with a 10% policy ordering 20 qualifying cases receives no threshold-based discount. At 50 eligible cases, the active policy can apply.

Code basis: backend/core/views_users.py; views_fleet.py: drivers_collection; order_lifecycle.py: _build_discount_breakdown_for_customer; api_constants.py

Inventory status and Needs Restocking

Presentation script

"The restocking status is calculated from available stock and the saved threshold. Zero available stock is Out of Stock. Positive stock at or below the threshold is critical. Stock slightly above the threshold is an early warning. The inventory list displays both warning levels as Needs Restocking."

How the logic works

- 1. The inventory API supplies sellableCases, excluding stock unavailable for allocation. The frontend prefers this value over a simple physical-stock subtraction.
- 2. The status checks run in this order: zero stock, critical, low warning, overstocked, then healthy.
- 3. For ordinary stock-in, the threshold becomes the greater of 1 and the integer part of 15% of the resulting physical case quantity. The overstock exception is explained on the next page.
- 4. Stock deductions do not continually lower that threshold. Keeping it fixed lets declining stock reach the warning boundary.

Internal level	Rule: A = available cases; T = threshold
Out of stock	A = 0
Critical	T > 0 and 0 < A <= T
Low warning	T > 0 and T < A <= 1.2 x T
Overstocked	After earlier checks, backend overstock flag is true
Healthy	None of the preceding conditions applies

Example to explain

With a saved threshold of 15: 0 available cases means Out of Stock; 1-15 is critical; 16-18 is low warning; 19 is Healthy unless an overstock rule applies. Critical and low both appear as Needs Restocking in the inventory list.

Code basis: src/lib/report-metrics.ts: getInventoryAvailableQty, getInventoryAlertLevel; src/components/portals/admin/sections/inventory-view.tsx; backend/core/views_inventory.py; views_stock.py

Stock-in and the Overstocked rule

Presentation script

"Stock-in records goods received into the warehouse. It adds a batch, increases inventory, and writes a transaction. Overstocking has a specific rule: both the latest stock-batch intake and the current available case quantity must reach ten times the saved threshold."

How the logic works

1. The operation validates the warehouse, product, quantity, batch information, expiry input, and warehouse capacity.
2. A successful stock-in increases physical quantity, records the restock time, and writes an IN transaction with before-and-after stock.
3. Normally the threshold is recalculated from the new total. If the incoming intake is at least 10 x the existing positive threshold, the old threshold is retained.
4. The backend overstock flag requires quantity minus reserved quantity $\geq 10 \times$ threshold AND latest IN stock_batch quantity $\geq 10 \times$ threshold.
5. Further stock-in is blocked while that flag is true. This is a current-state block, not a rule that rejects every large incoming batch in advance.

Example to explain

Suppose the threshold is 10, current quantity is 50, and none is reserved. Receiving 100 cases preserves threshold 10. The new quantity is 150 and the latest intake is 100, so both overstock tests pass. Further intake is blocked until the current flag clears.

Important implementation detail

The frontend has a fallback using 3 x threshold and seven days since restock when no backend flag exists. The normal inventory API supplies overstockedFlag, so its 10 x rule takes precedence.

Code basis: backend/core/inventory_overstock.py; views_stock.py: stock_batches_collection, stock_batches_bulk_collection; views_inventory.py: inventory_collection

Stock quantities and conversion formulas

Presentation script

"We keep physical cases, loose units, reserved stock, and sellable stock distinct. This avoids counting a loose bottle as a full case or offering goods already promised to another order."

How the logic works

1. Let Q = recorded cases, L = loose base units, U = units per case, R = reserved cases, and B = reserved base units. Physical base units are $Q \times U + L$.
2. The frontend fallback computes available physical cases = $\max(0, Q + \text{floor}(L/U) - R)$. Available base units = $\max(0, Q \times U + L - B)$. Available cases are the smaller of that case count and $\text{floor}(\text{available base units}/U)$.
3. When the API supplies sellableCases/sellableBaseUnits, those values take precedence. The server can exclude expired or unavailable batches and account for allocation records.
4. Loose remainder = $L \bmod U$. Complete loose sets should not be counted both as a displayed case and as leftover bottles.
5. Backend repacking can convert complete loose quantities within their original batch into cases. It preserves base-unit count and does not combine different expiry lots merely to make a case.
6. Quantities in CASE and BASE_UNIT transaction rows have different dimensions. Compare or total them only with the appropriate conversion or unit-aware report.

Example to explain

$Q=10$, $L=30$, $U=24$, $R=2$, $B=48$: physical units=270; available units=222; $\text{floor}(222/24)=9$. There are 9 available complete-case equivalents and a physical loose remainder of 6, before additional batch restrictions.

Code basis: src/lib/report-metrics.ts: getInventoryAvailableQty, getInventoryAvailableBaseUnits, getInventoryLooseRemainder;
backend/core/mixed_case.py: repack_batch_loose_stock

Stock-in validation, bulk intake, and retries

Presentation script

"Stock-in records a real receipt. Bulk intake applies that operation to several batches for one warehouse. Validation, capacity checks, and duplicate batch handling protect the inventory totals from accidental repeated submissions."

How the logic works

1. A single batch intake requires a positive quantity and expiry input; manufactured date, when supplied, must parse correctly. Warehouse and product selection are validated.
2. The bulk Add Stock flow requires each selected product to already exist in that warehouse inventory. It validates batch entries before processing their intake.
3. Before each positive stock increase, the warehouse capacity and current overstock block are checked. Intake updates quantity, the applicable threshold, and last-restocked time.
4. The batch and its IN transaction retain the intake reference and stock snapshots. Staff receive a stock-in notification when new batches are created.
5. For a retried bulk batch number, the code reuses the existing batch only when the inventory and quantity match; a conflicting use of the batch number returns a conflict.
6. For configured returnable packaging, intake may consume existing matching empty cases. Only available empties are consumed; the rest of the intake remains new stock.

Example to explain

Resending a matching bulk entry for batch B with 20 cases should reuse B rather than add another 20. Reusing B for another product is rejected.

Important implementation detail

A stock-in receipt is not a supplier purchase-order module. The reviewed routes record batches and inventory movements; they do not establish a separate procurement approval workflow.

Code basis: backend/core/views_stock.py: stock_batches_collection, stock_batches_bulk_collection; deposit_lifecycle.py: record_stockin_empty_consumption

Batch correction and manual inventory adjustment

Presentation script

"Correcting an existing batch is different from receiving a new shipment. The system records the correction as a new ledger event while preserving the original receipt."

How the logic works

1. Batch PUT accepts a non-negative quantity and can accept date edits. Changing quantity is blocked while the batch has active reservations.
2. Delta = new batch quantity minus previous batch quantity. A positive delta must pass overstock and warehouse capacity checks. No quantity/date change returns an unchanged result.
3. After a batch quantity correction, recorded case inventory is recalculated from positive batch quantities. An IN or OUT stock_batch_adjustment row records the absolute delta and stock snapshots.
4. Reducing a batch preserves the existing threshold. An increase may recalculate it; the code tests the new batch quantity against its overstock threshold rule.
5. Batch editing does not infer or consume returned empty containers. That behavior is distinct from the stock-in empty-consumption helper.
6. The manual inventory detail endpoint allows supplied quantity/reservation/loose fields; threshold is intentionally excluded. A physical case delta creates an inventory_manual_edit transaction with notes or a default reason.

Example to explain

Correcting a batch from 30 to 27 cases creates an OUT correction for 3 cases. The original receipt of 30 is not rewritten as if only 27 had ever arrived.

Important implementation detail

Manual corrections are available code paths, not permission to bypass stock reconciliation. Do not describe every field change as a new stock-in or promise a loose-unit audit row where this handler only logs the case delta.

Code basis: backend/core/views_stock.py: stock_batches_collection PUT; views_inventory.py: inventory_detail

Batches, transactions, and expired stock

Presentation script

"A batch identifies a particular stock intake. Inventory transactions explain why quantities changed. Expiry makes stock unavailable for selling, but it does not automatically erase the physical goods or their history."

How the logic works

1. Batch records include quantity, receipt/manufacture information, expiry, location label, and status. Allocation can choose eligible batches according to its stock policy.
2. Transaction records distinguish cases and base units and retain before-and-after snapshots where recorded. Reservations and physical OUT movements have different meanings.
3. Expired-stock removal requires a confirmed supplier return or disposal, positive whole quantity, a reason, and a request ID.
4. Removal is rejected if the batch is not expired, has active reservations, or lacks enough stock. Accepted removal reduces batch and physical inventory quantities.
5. An emptied batch remains in history and is marked depleted. Disposal stores listed-price loss snapshots; supplier return does not record the same disposal loss.

Example to explain

A warehouse may physically hold 20 cases, but if 5 are expired and the remaining 15 are otherwise free, only 15 are sellable. Disposing of the 5 expired cases then reduces physical stock to 15.

Important implementation detail

Disposal loss is based on the listed selling price snapshot. It is not a calculation of purchase cost or accounting profit.

Code basis: backend/core/views_inventory.py: resolve_expired_stock, disposed_stock_history, inventory_transactions_list; inventory_allocation.py: _sorted_batches_for_policy; views_stock.py

Expiry-health levels and disposal calculations

Presentation script

"Expiry health answers how close a batch is to its expiry date. It is separate from the inventory restocking badge, which answers how much stock is available. A batch can have plenty of cases and still be critical because it expires soon."

How the logic works

1. Expiry is checked against the actual expiry moment. Once expiry $\leq$ now, the health is Expired. Before that moment, the UI counts calendar days rather than rounding remaining hours.
2. Critical means 0-14 days left. Expiring Soon means 15-30 days. Healthy means over 30 days, or no valid expiry supplied to this display helper.
3. On the expiry calendar day, the UI can show Today while the batch is not yet expired. The backend stores date-only expiry at the end of the local day through its expiry validator.
4. Expired-stock actions accept SUPPLIER_RETURN or DISPOSAL, CASE or BASE_UNIT, a positive whole quantity, reason, and request ID. Active reservations must be released/reassigned first.
5. Disposal loss per case uses listed case price (or product price fallback). Loose disposal uses listed retail unit price, or case price divided by units per case. Loss = rounded unit price x disposed quantity.
6. Supplier-return rows retain their stock movement but use zero disposal loss. Empty batches remain as historical rows rather than being erased.

Batch health	Rule
Expired	Expiry moment has arrived/passed
Critical	Not expired; 0-14 calendar days left
Expiring Soon	15-30 calendar days left
Healthy	More than 30 days, or no valid expiry in display helper

Example to explain

A batch expiring in 10 calendar days is Critical even if 200 cases are available. Disposing of 3 cases valued at PHP 240 each records PHP 720 listed-price loss, not purchase-cost loss.

Code basis: `src/lib/stock-batch-health.ts`; `backend/core/views_inventory.py: resolve_expired_stock, disposed_stock_history`; stock expiry validator

Inventory alerts, transactions, and audit interpretation

Presentation script

"The inventory ledger explains each stock event, while notifications alert staff only when a boundary is crossed. A yellow badge does not mean an alert email must have been sent for every stock change."

How the logic works

1. Alert snapshots include physical cases, loose units, reserved cases/base units, threshold, and units per case. The signal compares old and new available complete-case counts.
2. The out-of-stock alert requires old availability > 0 and new availability = 0. Restock alert requires a positive threshold and old availability > current threshold >= new availability.
3. Initial inventory creation and repeated saves that do not cross a boundary do not issue these transition alerts. Dispatch occurs after transaction commit.
4. The UI early-warning band extends to 1.2 x threshold, while the signal threshold test does not. Backend alert counters also need not exactly match sellable-batch UI values.
5. Inventory transactions retain type, reference, unit, timestamp, and available snapshots of stock changes. RESERVE/UNRESERVE activity differs from physical IN/OUT.
6. Stock history filters include warehouse/product and date inputs where exposed. Disposal history adds batch identity, expiry/manufacture details, reason, actor, and listed-price loss basis.

Example to explain

Threshold 15: 19 -> 18 displays early warning; 16 -> 15 crosses the restock alert boundary; 15 -> 14 stays low without another crossing; 1 -> 0 triggers out-of-stock.

Code basis: backend/core/inventory_signals.py; views_inventory.py: inventory_transactions_list, _serialize_inventory_transactions_with_stock_changes; src/lib/report-metrics.ts

Catalog, cart selection, and Buy Again

Presentation script

"The customer catalog shows current products and availability. Adding to the cart prepares a request; it does not reserve stock yet. Only selected cart lines move into checkout, and Buy Again rechecks today's products instead of copying old availability."

How the logic works

1. The product loader fetches all catalog pages before publishing a complete new-session list. Catalog controls can search or narrow the displayed product choices.
2. Standard cart addition rejects out-of-stock products and caps quantity to available stock. Quantity updates clamp to availability; zero quantity removes the line. Selected lines can also be removed together.
3. Selected cart items drive subtotal, discount and deposit previews. Refreshed product data updates availability while retaining the customer's requested quantity so checkout can detect a new shortfall.
4. Only selected lines are removed after a successful order. Deposit-refund choices and the order note are cleared for the completed submission. Failed submission restores the cart and selection snapshot.
5. Buy Again refreshes the catalog, resolves product references, and skips missing/unavailable lines. A mixed-case repeat requests a fresh quote and checks component availability.
6. The mixed-case quote request is visible in the frontend but is not registered in the inspected Django URL list. Its full end-to-end operation is therefore not established by this review.

Example to explain

A past order contained five cases, but only two are available today. Buy Again cannot guarantee the old quantity at the old availability. The current cart and checkout checks still apply.

Code basis: `src/components/portals/customer/sections/cart/use-customer-cart.ts`; `sections/shared/products-api.ts`; `sections/orders/orders-api.ts`; `backend/core/urls.py`

Delivery address, map pin, and service area

Presentation script

"Delivery uses both the written address and a map pin. The interface keeps those fields together and restricts the delivery area to Silay and Talisay. A typed address alone is not enough to place an online order."

How the logic works

1. The address workflow hydrates saved profile fields, supports manual address parts and geocoding search, updates a chosen pin, and can save profile changes.
2. Before placing the order, the client requires a selected valid date not before today and both latitude and longitude. A failed location check reopens the address workflow.
3. The web/shared customer logic loads Silay/Talisay municipal polygons. If polygon loading fails, it uses the documented coarse bounding-box fallback.
4. The backend validates coordinate bounds and allowed city names Silay/Silay City/Talisay/Talisay City. A supplied province must normalize to Negros Occidental.
5. The checkout request snapshots the recipient, phone, composed address, city/province/ZIP/country, and coordinates onto the order.
6. A saved customer profile and an existing order's delivery-address snapshot are separate records; changing the profile should not be described as automatic rerouting of every existing order.

Example to explain

A customer enters Talisay but leaves the pin unset. The client blocks submission and asks for the location, allowing the driver and route planner to work with coordinates.

Important implementation detail

The backend bounds check and frontend polygon check are different implementations. Do not describe the server check as exact polygon validation.

Code basis: `src/components/portals/customer/sections/address/use-customer-address.ts`; `sections/cart/use-customer-cart.ts`; `shared/customer-logic/src/service-area.ts`; `backend/core/geo_utils.py`

Customer ordering, prices, and discounts

Presentation script

"The customer chooses products, quantities, a delivery date, and a delivery location. The server calculates the order from the product data and the customer's discount policy, then creates a purchase request for warehouse review."

How the logic works

1. Checkout requires item lines and checks delivery dates. A past delivery date is rejected; the address validation requires coordinates within the supported Silay/Talisay service validation.
2. The server normalizes standard and mixed-case items and calculates the product subtotal. It does not simply accept the client's displayed total.
3. The customer's configured discount must be active and the order must have at least 50 eligible case-equivalent quantities. Individual bottle quantities do not count toward that threshold.
4. The percentage discount is calculated from the subtotal and capped at the subtotal. Orders in this path have no tax or shipping charge.
5. The final amount is product subtotal minus discount, plus net deposits, minus validated deposit credit, with a minimum of zero.

Example to explain

Illustrative only: 50 qualifying cases with product subtotal PHP 10,000 and an active 5% discount produce a PHP 500 product discount. Before deposits or credits, the amount is PHP 9,500. At 49 eligible cases, that discount does not qualify.

Important implementation detail

Online checkout does not implement a payment-gateway flow here. The handler records payment status as pending; submitting a request is not evidence of a completed digital payment.

Code basis: backend/core/views_orders.py: orders_collection; order_lifecycle.py: _build_discount_breakdown_for_customer, _count_discount_eligible_cases, _create_order_from_checkout_payload

Online mixed-case composition and allocation

Presentation script

"A mixed case is one case made from component products. The system validates the composition, computes its price from those products, and reserves each component separately so one mixed case does not deduct a full case of every ingredient."

How the logic works

1. Online mixed-case logic sets a two-product limit and a per-product maximum of floor(case capacity / 2). Component quantities must meet the case-composition validation.
2. Positive whole-number checks reject booleans, non-finite values, zero/negative and fractional quantities in validated composition fields.
3. The online unit-price helper divides product price by units per case and retains six decimal places before money totals round to cents.
4. Component snapshots preserve product identity, quantity per case, case count, total base units, price, packaging, and deposit details on the order.
5. Allocation and availability use the actual component quantities across eligible stock batches. Repacking preserves source-batch units and expiry identity.
6. Retail has its own mixed-case rules. It requires exactly two compatible glass-bottle products and quantities that fill the selected cases; do not assume every online constraint is identical in POS.

Example to explain

For a 24-unit online case, the two-product half-case cap is 12 each. Three such cases require 36 units of product A and 36 of B; they do not require three whole cases of each product.

Important implementation detail

The source includes checkout normalization and the frontend builder, but `/api/mixed-cases/quote` is absent from the inspected active route registry. This is a documented integration gap, not a verified working quote service.

Code basis: `backend/core/mixed_case.py`; `retail_pos.py`: mixed-case validation;
`src/components/portals/customer/sections/cart/mixed-case-builder-dialog.tsx`

Purchase requests and approval

Presentation script

"Submitting a request asks the warehouse to fulfill an order. A request that can be fulfilled reserves stock immediately. Warehouse approval creates the purchase order identity and records who approved it. A request with insufficient stock can remain visible for review, but approval must still pass the stock checks."

How the logic works

1. Checkout creates a pending purchase request. It attempts stock reservation inside a transaction.
2. If that reservation fails specifically because stock is insufficient, the reservation attempt is rolled back while the request remains available for review.
3. Only warehouse staff can approve pending requests. Approval checks or creates reservations without reserving an already-reserved request twice.
4. Approval assigns the PO number, retains or creates the PR number, sets the request to APPROVED, and records approver identity and time.
5. Rejection and cancellation require a reason and release reservations. Unapproved requests with a past delivery date can be expired by the request-expiry helper.

Example to explain

There are 100 sellable cases. A fulfillable request for 20 reserves them, so new buyers see 80 available while the 100 physical cases remain. Approval keeps that reservation instead of reducing availability by another 20.

Important implementation detail

Request submission, approval, and physical delivery are separate events. A PR is not automatically a delivered sale.

Code basis: backend/core/views_orders.py: orders_collection, order_status_update; inventory_allocation.py: _reserve_order_inventory; order_fulfillment.py: _expire_past_delivery_purchase_requests

Request expiration, duplicate submission, and document identity

Presentation script

"A request, purchase order, and delivery are related but retain different identities. The system handles repeat submissions and expired delivery dates so a stale or repeated request does not silently become a second valid fulfillment."

How the logic works

1. The customer creates a stable request ID for an unchanged checkout payload. A changed payload receives a new ID; a successful submission clears the pending client request reference.
2. The backend checks request identity before creating an order. An uncertain response should be retried through the same workflow rather than blindly creating a new request.
3. A pending request has request_status PENDING_APPROVAL. Approval issues a real PO number and stores approver/time metadata; retaining a PR number does not make it a second sale.
4. Request-expiry handling finds PENDING/PENDING_APPROVAL requests whose delivery date is before local midnight today. It changes both statuses to CANCELLED, leaves PO stage empty, and records System as the actor.
5. Expiration releases reservations in the cancellation transaction, synchronizes purchase documents, and sends a customer notification.
6. This expiration helper is called from operational reads/updates. Its presence does not prove a separate always-running midnight scheduler exists.

Example to explain

A request planned for yesterday remains unapproved. When the expiry helper processes it, it is cancelled with Delivery date expired before approval and its stock reservation is released.

Code basis: src/components/portals/customer/sections/cart/use-customer-cart.ts; backend/core/views_orders.py; order_fulfillment.py: _expire_past_delivery_purchase_requests; purchase_documents.py

Purchase-order status and cancellation

Presentation script

"After approval, the purchase order moves through processing, dispatch, and delivery. Each status represents an operational event. Assigning an approved purchase order to a trip can start processing, and starting the trip changes the order to Out for Delivery."

How the logic works

1. Typical path: PENDING -> APPROVED -> PREPARING -> OUT_FOR_DELIVERY -> DELIVERED. PREPARING is shown to users as Processing.
2. Manual processing requires an approved request and a valid delivery date. Trip assignment also moves qualifying APPROVED or RESCHEDULED purchase orders into Processing.
3. Out for Delivery is set by trip start; the generic status endpoint rejects a manual request to set it.
4. Rescheduling requires a valid non-past date and removal from an active trip before the manual reschedule proceeds.
5. Cancelled or rejected orders release reservations; delivered orders cannot move to another status through the status-transition endpoint.

Example to explain

An approved PO scheduled for tomorrow is assigned to a planned trip. It becomes Processing. When the driver starts that trip on its scheduled day, it becomes Out for Delivery.

Important implementation detail

The customer-cancel endpoint specifically blocks PREPARING, DELIVERED, and REJECTED and requires a reason. Its current guard does not explicitly block OUT_FOR_DELIVERY; do not describe a stricter cancellation rule as universally enforced.

Code basis: backend/core/models.py: OrderStatus; views_orders.py: order_status_update; views_trips.py: _start_processing_for_trip; views_customer.py: customer_order_cancel

Order status transition reference

Presentation script

"Status changes are validated against the current state. This table shows the generic order status endpoint's allowed next-state sets, with the additional approval and trip-start gates applied on top."

How the logic works

- 1. A pending approval must be approved before Processing. Only warehouse staff can approve a pending request.
- 2. OUT_FOR_DELIVERY appears in the transition set but the generic endpoint rejects manually setting it; trip start performs dispatch.
- 3. Repeated same-state updates can return the current record; Delivered, Rejected and Cancelled do not have onward transitions in this table.
- 4. Manual reschedule requires a valid date and removal from an active delivery trip. Rejection/cancellation needs a reason.

Current order state	Allowed next-state set before extra guards
PENDING	APPROVED, PREPARING, REJECTED, CANCELLED
APPROVED	PREPARING, RESCHEDULED, REJECTED, CANCELLED
PREPARING	RESCHEDULED, OUT_FOR_DELIVERY, REJECTED, CANCELLED
RESCHEDULED	PREPARING, OUT_FOR_DELIVERY, REJECTED, CANCELLED
OUT_FOR_DELIVERY	DELIVERED, REJECTED, CANCELLED
DELIVERED / REJECTED / CANCELLED	No different next state

Example to explain

Attempting APPROVED -> DELIVERED through the generic status endpoint is not in the allowed set. Normal fulfillment must progress through its operational steps.

Code basis: backend/core/views_orders.py: order_status_update (allowed_transitions and preceding guards)

Reservations, allocation, and mixed cases

Presentation script

"Inventory allocation connects an order to the stock that will fulfill it. Reservations prevent the same available stock from being promised twice. Mixed cases additionally track each component product so that the correct bottles are reserved and deducted."

How the logic works

1. Allocation uses inventory and batch eligibility, including expiry and unavailable batch states. FEFO means First Expired, First Out; other supported allocation policy branches are kept separately.
2. A reservation raises reserved counters or creates reservation records. It reduces availability before physical stock is consumed.
3. Delivery finalization consumes the applicable stock and settles reservations. Cancellation releases reservations without treating unshipped stock as new stock-in.
4. Case quantity and base-unit quantity are tracked separately. Whole loose sets can contribute to displayed cases; leftover bottles remain a loose remainder.
5. Mixed-case lines store component products, quantities, pricing, and deposit details. Allocation must have sufficient units of each component.

Example to explain

With 10 cases plus 30 loose bottles, and 24 bottles per case, there are 11 complete-case equivalents and 6 loose bottles before reservations or batch restrictions. A mixed case consumes its component quantities rather than 1 case of every selected product.

Important implementation detail

Some orders have allocations across warehouses or trip legs. Completion must account for the relevant allocations, not merely assume that one stop represents every unit in an order.

Code basis: `backend/core/inventory_allocation.py`; `mixed_case.py`; `order_fulfillment.py`; `src/lib/report-metrics.ts`: `getInventoryAvailableQty`, `getInventoryLooseRemainder`

Driver profiles, service areas, and vehicle pairing

Presentation script

"A driver needs an eligible profile and a compatible assigned vehicle before dispatch. Account activation, operational availability, license qualification, service area, and current trip are separate checks."

How the logic works

1. The assignment blocker rejects inactive accounts, On Leave/Inactive driver status, and incomplete phone/license-number/restriction/expiry information.
2. License-number format and restriction-code values are validated. An expired license blocks assignment. The project policy requires code C for trucks and A1 for tricycles.
3. Warehouse staff manage vehicles and operational driver updates. Administrators assign driver service areas through the separate authorized action.
4. A vehicle requires a model on creation, plus plate and type. Duplicate plates are checked case-insensitively. One driver cannot be assigned to a second vehicle through these assignment checks.
5. Removing a driver from a vehicle is blocked while the driver has an IN_PROGRESS trip. Changing license codes cannot leave an incompatible current vehicle assignment.
6. Completed delivery counts are derived from completed DELIVERY drop points and are not manually editable counters.

Example to explain

An Active driver with no license expiry date is still incomplete. A driver with only A1 does not satisfy this project's truck-code check.

Important implementation detail

The license rules here describe the application's implemented policy, not independent legal advice about vehicle licensing.

Code basis: backend/core/driver_assignment.py; driver_license.py; views_fleet.py; views_users.py

Drivers, vehicles, and trip planning

Presentation script

"Warehouse staff create a trip by selecting delivery work, a driver, and a vehicle. The system checks whether the work and resources are eligible, then records the delivery stops and their assignments."

How the logic works

1. Pending, delivered, cancelled, and rejected orders are blocked from trip assignment.
2. The assigned warehouse must be within the staff member's scope. Driver and vehicle records must exist, and the driver must satisfy the vehicle-license checks.
3. The selected vehicle must be assigned to the selected driver and available for the new trip. Service-area checks must also pass.
4. The assigned load is calculated using product quantities and weight. A load exceeding the vehicle rule is rejected.
5. A newly created trip is planned. Planned trips can be edited, but protected terminal stops cannot simply be removed. Trip requests use request IDs to handle duplicate submissions.

Example to explain

If the selected products weigh more than the vehicle can carry, trip creation fails with the overload message. Staff must change the assignment or load before dispatch.

Important implementation detail

Driver account activation, driver operational availability, vehicle availability, and trip status are different fields. One does not replace all the others.

Code basis: backend/core/views_trips.py: trips_collection, trip_detail, _trip_order_status_error; inventory_allocation.py: _calculate_order_item_weight, _vehicle_overload_message; fleet_sync.py

Upcoming deliveries, route grouping, and trip changes

Presentation script

"Planning groups eligible delivery work by day and warehouse. Staff select the orders that fit a driver, service area, and vehicle. Editing a planned trip changes assignments without pretending that the goods have already been dispatched."

How the logic works

1. The Create Trip preview requests the next five days by default. The planning date is the order delivery date, falling back to creation date when no delivery timeline date exists.
2. Pending/unapproved and closed order states are excluded. Orders already attached to active drop points in the scoped warehouse are excluded from that available planning pool.
3. Warehouse load uses that warehouse's item allocations when present. Total cases and weight are computed from the quantities assigned to this leg.
4. The route-plan helper enriches the selected order sequence with Haversine segment distances. That calculation does not itself prove a shortest-route optimization.
5. Only planned trips can be edited/deleted through the trip detail handler. Edits revalidate driver/vehicle, eligible orders, service area, warehouse allocations and overload. A trip must keep at least one stop.
6. Unassigning item allocations requires a planned trip and valid order/warehouse/item IDs. Removing a planned trip does not mean deleting the original purchase order.

Example to explain

An order has allocations in two warehouses. The trip planner should count the selected warehouse leg's assigned quantity, rather than loading the full order from both warehouses.

Important implementation detail

Saved-route frontend calls reference `/api/trips/saved-routes`, which is absent from the reviewed route registry. Planned trips are routed; a separate saved-route service is not confirmed.

Code basis: `backend/core/views_trip_planning.py`; `views_trip_ops.py`: `trips_route_plan`; `views_trips.py`: `trip_detail`, `trip_unassign_items`; `src/components/portals/warehouse/sections/trips/use-warehouse-route-planning.ts`

Trip start, delivery proof, and closure

Presentation script

"The driver confirms the load before starting the trip. During delivery, the driver updates the stop and uploads proof of delivery. Closing a trip means its stops have been resolved; it does not mean every stop necessarily succeeded."

How the logic works

1. Start requires a planned trip on its scheduled date, load confirmation, appropriate driver ownership, a valid license check, and a usable vehicle.
2. A driver cannot start a second trip while another trip is in progress. Vehicle maintenance and out-of-service states block departure.
3. Trip start sets IN_PROGRESS, records its start, and updates applicable orders to OUT_FOR_DELIVERY.
4. Completing a delivery stop requires a POD photo. Completion then processes the order delivery and the applicable inventory and empty-container effects.
5. Failed or deferred stops have separate handling. Same-day deferral can return the stop to pending; later rescheduling updates the order schedule/state.
6. Trip completion requires at least one drop point and no outstanding unresolved drop points. The end time and resource status are updated.

Example to explain

If two stops are delivered but the third is still pending, the trip cannot close. The driver must resolve the third stop through the appropriate delivery, failure, or rescheduling workflow.

Important implementation detail

An uploaded photo is proof required by this workflow; arrival on the map alone does not automatically confirm delivery.

Code basis: backend/core/views_trip_ops.py: trip_start, trip_drop_point_update, trip_complete; order_fulfillment.py; fleet_sync.py

Scheduled day, vehicle state, and explicit trip completion

Presentation script

"A trip can start only on its scheduled day. While it is running, its vehicle is in use. Finishing the last delivery stop does not automatically finish the trip; the driver still confirms trip completion."

How the logic works

1. Trip scheduled day is the earliest delivery date among its order drop points. `planned_start_at` is the fallback for records without those dates.
2. A PLANNED trip whose day has passed is overdue and cannot be started normally. A future planned trip also cannot be started before its scheduled day.
3. After load confirmation and the other start checks, the trip becomes IN_PROGRESS and the vehicle status synchronizes to IN_USE.
4. Vehicle sync preserves manually assigned MAINTENANCE and OUT_OF_SERVICE. Without an active trip, an IN_USE vehicle returns to AVAILABLE.
5. Terminal drop-point states used by execution are COMPLETED, FAILED, SKIPPED and CANCELLED. The stored `completed_drop_points` counter counts resolved terminal stops in this path.
6. The final stop update keeps an unclosed trip IN_PROGRESS when it has a start time. Explicit `trip_complete` checks all stops are resolved, saves `actual_end_at`, and synchronizes the vehicle.

Example to explain

A trip with two successful deliveries and one terminal failure can be ready for closure, but its successful delivery count is not three. Until Complete Trip is confirmed, it can still be the driver's active trip.

Code basis: `backend/core/fleet_sync.py`; `views_trip_ops.py`: `trip_start`, `trip_drop_point_update`, `trip_complete`; `views_driver.py`: `_driver_trip_stats`

Failed delivery, same-day retry, and rescheduling

Presentation script

"A failed attempt can mean a retry, a later delivery date, or a terminal cancellation. The chosen outcome determines whether the order keeps its stock reservation and whether the stop still belongs to the driver's remaining work."

How the logic works

1. For FAILED with a same-day retry request, the stop is changed back to PENDING and moved to the end of the trip sequence. It is still unresolved work.
2. For a failed delivery with a later reschedule request, the order becomes RESCHEDULED and its timeline gets the selected date. It can return to the route-planning pool.
3. SKIPPED/CANCELLED and FAILED without rescheduling release order reservations. A rescheduled failure ordinarily keeps the reservation unless the explicit releaseInventory flag is set.
4. Terminal outcomes that are not reschedules change the order to CANCELLED, set PO stage CANCELLED, and record actor, time and cancellation reason.
5. The stop retains its failure/result information and the workflow emails the customer about the outcome where transport is available.
6. The request parser accepts explicit ISO reschedule dates and today/tomorrow windows. Do not assume every validation in the manual staff-reschedule endpoint is identical to the driver failure endpoint.

Example to explain

The customer is temporarily away: retry today puts the stop last and keeps the delivery active. Choosing terminal cancellation releases the reserved goods for other work instead.

Code basis: backend/core/views_trip_ops.py: trip_drop_point_update; src/components/portals/driver/sections/trips/failed-delivery-dialogs.tsx

POD camera, uploads, and private evidence

Presentation script

"The driver captures and uploads delivery evidence before confirming the stop. Product images are public catalog assets, but delivery photos and replacement evidence are served through authenticated media checks."

How the logic works

1. Completing a stop requires a non-empty POD photo reference. Camera controls obtain a capture and the upload result is passed into the completion workflow.
2. POD upload is driver-only. If overlay metadata is supplied, the server validates it and burns the authenticated driver's name and overlay information into the image.
3. Images are optimized server-side. The upload helper uses a 10 MB image limit and a 50 MB non-image limit; replacement evidence accepts image/video MIME types.
4. Uploaded names use random identifiers. Configured object storage is preferred; local media storage is the fallback when that storage is not configured.
5. Non-catalog files return an `/api/media/` path so ownership/role checks apply at retrieval. Product images can be served publicly for the catalog.
6. Storage/upload errors produce an error result. A failed upload should not be presented as a completed proof or a confirmed delivery.

Example to explain

The driver captures a photo but storage returns an error. The stop still needs a valid uploaded photo; seeing a local camera preview is not the same as saving POD.

Code basis: `backend/core/views_media.py`; `pod_overlay.py`; `src/components/portals/driver/sections/trips/use-trip-pod-camera.ts`; `pod-camera-dialogs.tsx`; `views_trip_ops.py`

Live tracking, navigation, and ETA

Presentation script

"Tracking uses the driver's recorded location to show delivery progress. The driver navigation screen requests a road route and directions. Estimated arrival times are estimates derived from location and routing information, so they can change while the trip progresses."

How the logic works

1. Driver location records provide coordinates, timestamps, and optional accuracy, heading, and speed. Trip data returns the latest relevant location.
2. Customer tracking selects the customer's orders and associated trip, driver, destination, and delivery status.
3. The customer tracking API estimates ETA only for Out for Delivery orders with driver and destination coordinates.
4. That API uses straight-line Haversine distance divided by speed: positive GPS speed is converted to km/h, otherwise 24 km/h is used. Speed is clamped to 10-70 km/h; ETA is rounded up with a one-minute minimum.
5. The driver navigation hook requests OSRM driving routes, turn steps, and alternatives. Its road-route behavior is separate from the simple customer API estimate.

Example to explain

For a straight-line remaining distance of 12 km and a speed assumption of 24 km/h, the customer API estimates 30 minutes. Road layout, stops, and changing speed can make actual arrival different.

Important implementation detail

The inspected distance helper measures segments in the supplied order. Do not claim that every route is globally optimal or that the customer ETA includes live traffic.

Code basis: backend/core/views_driver.py; views_customer.py: customer_tracking; geo_utils.py: _haversine_km, _compute_order_distances; src/components/portals/driver/sections/trips/use-trip-navigation.ts

Location freshness, retry behavior, and trip history

Presentation script

"Location and delivery screens must distinguish the latest confirmed server state from a pending request or cached snapshot. The driver portal refreshes its trips and history, and temporary network failures are retried selectively."

How the logic works

1. Location uploads queue the newest position and retain timestamps so delayed callbacks after resume do not rewind the displayed truck to an older sample.
2. The driver portal uses device/native location when available, with permission and accuracy handling. No location permission or no fresh fix limits what can be shown.
3. Automatic write retries cover transport/timeouts, HTTP 408, HTTP 429, and server errors. Validation failures should be corrected rather than retried indefinitely.
4. The write-retry delay increases exponentially from about one second with a 15-second cap. A successful-looking optimistic update is later reconciled with refreshed server data.
5. Driver Home counters come from all the driver's trip records, not one displayed page. The History view presents historical assignments separately from active work.
6. Refresh, cached reads and selective retries do not prove that every transaction can be completed offline. The reviewed code must still receive an accepted backend result for confirmation.

Example to explain

If mobile connectivity drops after a request was sent, a retry can recover the response. A missing required POD photo is a validation problem, not a reason to keep retrying the same invalid payload.

Code basis: `src/components/portals/driver/sections/layout/portal-state.ts`; `sections/trips/trip-detail-retry.ts`; `DriverPortal.tsx`; `backend/core/views_driver.py`

Replacement requests and decisions

Presentation script

"A customer can report an issue with delivered goods and request a replacement. The system validates the delivered items, quantity, reason, and evidence. Admin reviews the claim, while warehouse staff handle processing and scheduling after approval."

How the logic works

1. The source order must belong to the customer and be delivered. Where a delivery timestamp is available, requests are limited to three days after delivery.
2. The request needs positive whole quantities, valid source items or mixed-case components, a reason, and at least one evidence file.
3. Quantities cannot exceed the remaining delivered source allocation. Duplicate active requests and completed-request restrictions are checked.
4. Admin actions include Under Review, Approved, or Rejected. Rejection requires notes. Warehouse staff may move Approved to In Progress and schedule its delivery with manual confirmation.
5. The customer may cancel only a Pending replacement. Delivery-linked progress determines completion, and outstanding replacement quantities prevent premature completion.

Example to explain

A customer reports two damaged bottles from a delivered order and attaches evidence. Admin approves the request; warehouse staff start processing and schedule the replacement delivery. Approval alone does not mean the new goods have already arrived.

Important implementation detail

Replacement statuses and ordinary purchase-order statuses are separate lifecycles. A replacement delivery also has its own linked fulfillment record.

Code basis: backend/core/views_customer.py: customer_replacements, customer_replacement_cancel; views_orders.py: replacement PATCH workflow; replacement_services.py; order_fulfillment.py: _advance_replacement_line_progress_on_delivery

Replacement scheduling, line progress, and closure

Presentation script

"The replacement request describes the claim, and its linked delivery performs the fulfillment. Admin approval is a decision; warehouse processing and actual replacement delivery are later events."

How the logic works

1. Customer claims reference delivered source items and valid mixed-case components. Whole quantities, per-line reasons, evidence and source-allocation limits are checked.
2. Admin may set Under Review, Approved or Rejected through the replacement decision workflow. Rejection requires notes. Warehouse staff may start Approved -> In Progress.
3. Creating a replacement delivery requires In Progress eligibility, a scheduled date, and manual schedule confirmation. Only warehouse staff may schedule or reschedule it.
4. The request stores scheduling and status-timeline metadata. A later schedule update must meet its reschedule eligibility and non-past-date checks.
5. Delivery advances the replacement line quantities. Completing/resolving a request is rejected if outstanding replacement quantities remain.
6. Cancelling/rejecting a linked replacement delivery synchronizes the replacement request to a closed outcome rather than leaving a contradictory active request.

Example to explain

If a claim requires four units and only two have been fulfilled, the remaining two are still outstanding. The request should not be marked fully completed merely because one linked stop finished.

Important implementation detail

Legacy states such as REPORTED and NEEDS_FOLLOW_UP remain supported in model/helpers; they are not all mandatory steps in every customer claim.

Code basis: backend/core/views_orders.py: replacement PATCH handling; replacement_services.py; order_fulfillment.py: _advance_replacement_line_progress_on_delivery; views_customer.py

Returnable bottles, cases, and deposits

Presentation script

"Returnable packaging is tracked separately from beverage stock. The system calculates deposits from bottle and case configuration. A customer's declared empties and the containers actually collected by the driver are different events."

How the logic works

1. A full-case deposit is bottle deposit x bottles per case, plus the physical-case deposit. Loose bottles use the per-bottle deposit.
2. The customer empty-recording feature requires a returnable product and purchase history, and checks allowed quantities. It records a declaration and related balance adjustment.
3. Driver collection verification settles the actual returned containers. A declaration by itself is not proof that a physical return took place.
4. If delivery is finalized without a driver count, declared empties are recorded as a pending return and deposits stay charged until reconciliation.
5. Warehouse empty stock supports explicit return movements. Returnable stock-in can consume matching available empty cases; insufficient empties do not block the rest of the new stock.

Example to explain

Illustrative deposits: PHP 2 per bottle, 24 bottles per case, plus PHP 20 for the case gives PHP 68 per full case. Two full cases and three loose bottles give $(2 \times 68) + (3 \times 2) = \text{PHP } 142$.

Important implementation detail

For stock-in of 4 returnable cases with only 2 matching empty cases available, the helper consumes those 2 empties and accepts the remaining intake as ordinary new stock.

Code basis: backend/core/deposit_math.py; deposit_lifecycle.py: finalize_order_deposits_on_delivery, record_stockin_empty_consumption; empties_verification.py; views_bottles.py; views_inventory.py: record_returned_empty_containers

Deposit credit on a purchase order

Presentation script

"A deposit credit uses eligible product-specific empty containers to reduce an order's amount due. It is recorded as a claim for physical collection; it is not an unrestricted cash refund."

How the logic works

1. The customer must own the order. Adding credit after checkout requires a real PO number, an undelivered/non-cancelled/non-rejected order, and no completed bottle-return collection for that order.
2. Each claim identifies product and container type, with explicit case/bottle quantities. Duplicate product lines, mismatched packaging, no eligible balance, and zero quantities are rejected.
3. The server prices selected quantities using deposit configuration and verifies that they do not exceed available eligible empties or the order's amount due.
4. The accepted claim reduces total_amount with a floor of zero. Pending claim quantities prevent the same eligible balance from being promised repeatedly.
5. When requestId is supplied, an already-committed refund request for that same order returns its prior applied amount. Reusing the ID for another order is rejected.
6. Collection later settles, partially settles, or rejects claims using the driver's actual count. Payment status is not the eligibility gate for this credit endpoint; delivery/collection state is.

Example to explain

An eligible credit of PHP 100 can reduce a PHP 600 undelivered PO to PHP 500. A request for PHP 700 of selected empties is rejected because it exceeds the amount due.

Code basis: backend/core/views_customer.py: customer_order_deposit_refund; order_lifecycle.py: _create_deposit_refund_claims; models.py: OrderDepositRefundClaim, OrderDepositRefundRequest

Collected empties, shortages, and settlement states

Presentation script

"The driver records what was actually collected. The system compares that count with the declaration, settles the eligible credit once, and charges back any shortage that reduced the order without being physically returned."

How the logic works

1. Submitted collection lines are grouped by container type. For a declared quantity, settlement collection is capped at that declaration; extra containers are not an unlimited refund on the order.
2. Short quantity = $\max(0, \text{declared quantity} - \text{collected quantity})$. Shortfall value uses the declared deposit value multiplied by short quantity / declared quantity, rounded to cents.
3. An existing BottleReturn for the order is returned as already recorded, preventing this settlement path from refunding the same collection twice.
4. The code records reusable collected containers through the bottle-return service. If declared containers have no collection, it creates the appropriate uncollected-return record.
5. Pending credit claims become SETTLED when fully collected, PARTIAL when some are collected, or REJECTED when none are collected. Collected claim value is full or proportional to requested quantity.
6. Collected amounts reduce the applicable deposit balance. The stop links to its bottle return, and any shortfall invokes the charge-shortfall operation on the order/customer records.

Example to explain

Ten declared containers carry PHP 100 credit but only eight are counted. The proportional shortfall is PHP 20 and the collected share is PHP 80, subject to the grouped claim allocation.

Important implementation detail

A customer declaration, refundable balance, returned-container stock, and collected-on-delivery settlement are separate concepts even when the screen groups them under Empties and Deposits.

Code basis: backend/core/empties_verification.py: record_collected_empties, _charge_shortfall; deposit_lifecycle.py; rgb/services.py

Warehouse empty cases and explicit returns

Presentation script

"The warehouse tracks returned empty containers independently of full beverages. It can record an explicit empty return, and stock-in can reuse available matching empty cases without inventing extra empty stock."

How the logic works

1. Empty inventory is derived from the product/warehouse returnable-container records and transaction history. This is not the same quantity as sellable full cases.
2. The explicit return endpoint requires inventory ID, CASE or BOTTLE unit, and a positive whole quantity. The product must be returnable and within the staff warehouse scope.
3. An explicit returned quantity cannot exceed available empty stock. The resulting record reduces the balance of empties held for the product as that outward return is recorded.
4. Returnable stock-in can consume matching available empty cases. A prior consumption record for the batch is reused/updated so intake replay does not consume the same empties twice.
5. When received cases exceed available empties, only available empties are consumed. The remaining intake is ordinary new stock; it is not blocked solely by missing empties.
6. Correcting batch quantities is a separate path and does not infer empty-container consumption. Some old UI copy says stock-in never consumes empties, but the active stock-in helper does under its packaging conditions.

Example to explain

The warehouse holds 12 matching empty cases. It explicitly returns 4, leaving 8. A later stock-in can consume up to the available matching balance; it cannot consume nonexistent empties.

Important implementation detail

The routed GET /api/bottle-returns currently returns an empty list. A full generic return-history screen should not be claimed from that endpoint alone.

Code basis: backend/core/views_inventory.py: empty_case_inventory, record_returned_empty_containers; deposit_lifecycle.py: record_stockin_empty_consumption; src/components/portals/warehouse/sections/inventory/empty-bottles-view.tsx

Retail point of sale

Presentation script

"Retail handles immediate counter sales. Staff select an existing customer or a walk-in customer, build the cart, and request a current quote. Completing the sale immediately consumes stock and creates the retail transaction record."

How the logic works

1. Supported sale modes include loose units, cases, and mixed cases. Every selected product and requested quantity is checked against current sellable inventory.
2. Retail mixed cases require exactly two different eligible glass-bottle products, matching case capacity and compatible packaging. Component quantities must fill the selected cases correctly.
3. The server returns a quote fingerprint. If pricing, packaging, deposit, product, or inventory configuration changes, the sale must obtain a fresh quote.
4. An idempotency key identifies the submission so a retry can return its existing sale instead of completing another sale.
5. Completion creates a delivered Order with the RETAIL_POS channel and a completed RetailSale record, then records deposits/returns and consumes reservations.
6. Cancellation restores consumed inventory and reverses the related deposit/return effects. Accepted empties require physical restoration/correction confirmation.

Example to explain

A cashier sells six loose bottles. The sale consumes six base units when completed. It does not need a purchase-request approval and delivery trip before reducing stock.

Important implementation detail

The inspected retail completion path stores discount = 0. Do not assume the online 50-case client-discount rule automatically applies to retail.

Code basis: backend/core/retail_pos.py: quote_retail_cart, create_retail_sale, cancel_retail_sale; views_retail.py

Retail quote, receipt, cancellation, and units

Presentation script

"The retail quote is the authoritative preview for a counter sale. Once accepted, the sale stores product and packaging snapshots, consumes the exact units, and can be reviewed from its transaction record."

How the logic works

1. Existing customers must be available; walk-in details are validated where supplied. The warehouse-scoped product list provides the current sale options.
2. Loose, case, and mixed-case rows convert to the relevant base quantities. The cart quote checks combined requested quantities against the available stock, not just each line in isolation.
3. The quote fingerprint must match at completion. A changed product, price, packaging, deposit, or inventory configuration requires a refreshed quote.
4. Completion requires an idempotency key. It stores a RetailSale header with creator and transaction number plus item/component price, packaging, unit and deposit snapshots.
5. It records returnable deposit/empty effects, reserves the needed units using FEFO, and consumes them in the completed sale workflow. Retail discount is zero in the inspected creation code.
6. Cancellation restores consumed stock and reverses deposit/return effects. If accepted empties were involved, staff must confirm their physical restoration/correction before cancellation.

Example to explain

A two-line cart requesting 5 and 7 units of the same available product must be checked against 12 combined units. Treating each line as independently below stock would promise too much.

Important implementation detail

Review/receipt values come from the saved transaction. They should not silently change to current product prices after the sale.

Code basis: backend/core/retail_pos.py: quote_retail_cart, create_retail_sale, serialize_retail_sale, cancel_retail_sale; views_retail.py

Dashboards and reports

Presentation script

"Dashboards summarize recorded operations, while reports let staff examine those records using filters. A displayed total depends on its source, date range, status, and warehouse scope, so two different report cards do not always count the same population."

How the logic works

1. The dashboard API counts orders by status, active trips, clients, vehicles, and rated feedback. Its totalRevenue sums total_amount for delivered orders.
2. Average rating equals the sum of non-null ratings divided by their count. Unrated feedback does not add a rating to that denominator.
3. The dashboard API low-stock count uses physical quantity $\leq$ threshold + reserved quantity. It does not use every sellable-case and early-warning rule from the inventory badge.
4. Driver dashboard counters use all of that driver's trips, not just one page. Completed Today counts completed trips scheduled for today.
5. Report tabs include purchase requests, purchase orders, transaction records, logistics, replacements, retail sales, top clients, warehouse, inventory, and feedback. Filters narrow the report data.

Example to explain

With threshold 15 and 18 available cases, the inventory list can show Needs Restocking because of its 20% warning band. A dashboard count restricted to at-or-below-threshold stock may not count the same item.

Important implementation detail

Delivered-order total value is not automatically profit or confirmed cash collection. Deposits and channel selection must be understood before interpreting a revenue total.

Code basis: backend/core/views_dashboard.py: dashboard_stats; views_driver.py: _driver_trip_stats; src/components/portals/admin/sections/reports-view.tsx; src/lib/report-metrics.ts

Warehouse and driver dashboard calculations

Presentation script

"Each dashboard summarizes the records relevant to its role. Warehouse staff see their warehouse workload and capacity, while drivers see their own scheduled and active trips."

How the logic works

1. Warehouse stock cards use `summarizeStockHealth` on scoped inventory. Below-threshold count is available $\leq$ threshold when the threshold is positive, not the entire early-warning band.
2. Warehouse pending workload includes `PENDING`, `APPROVED`, `PREPARING` and `RESCHEDULED`. Active trips use the active-trip status helper.
3. Capacity helper usage = $\text{stored case quantity} / \text{configured capacity} \times 100$, displayed with a cap of 100%. Status is Healthy below 55%, Moderate from 55%, High from 75%, Critical from 90%.
4. When no positive capacity exists, that display helper substitutes $\max(1000, \text{used} + 250)$. This is a UI fallback, not the backend hard-capacity rule.
5. Utilization trend reconstructs estimated past stock by subtracting subsequent net IN/OUT from today's stock; when transactions are absent it falls back to intake batches.
6. Driver Today counters use scheduled day, excluding cancelled trips. Completed Today means completed trips scheduled today; overdue means a planned trip whose scheduled day passed.

Example to explain

At 800 cases of configured capacity 1000, the shared warehouse capacity helper shows 80% and High. Reserved cases still occupy the warehouse and remain in used storage.

Important implementation detail

Some warehouse activity-display entries use fixed relative time strings such as 2 mins ago. Treat these as display copy, not verified audit timestamps.

Code basis: `src/components/portals/warehouse/sections/dashboard/use-warehouse-dashboard-stats.ts`; `src/lib/report-metrics.ts`: `buildWarehouseCapacitySummary`, `buildUtilizationTrend`; `backend/core/views_driver.py`

Report loading, date filters, export, and recognition

Presentation script

"Reports load the collections they actually need. Filters decide the population, and exports use the prepared report rows. A request value, delivered revenue, and a payment label are separate measures."

How the logic works

1. The data plan maps each tab to its datasets: PR/PO use orders; logistics uses trips/drivers/warehouses; inventory also uses transactions, batches and sales; feedback uses feedback/orders/trips.
2. Reports offer calendar presets and, where exposed, inclusive custom date windows. Different reports date records by submission, approval, or their own event basis.
3. The shared recognition helper normalizes DELIVERED, COMPLETED, FULFILLED and legacy ARRIVED to Delivered; cancelled/rejected/failed/skipped aliases normalize to Cancelled. Other values are Pending.
4. Recognition reads status, then stage, then retailStatus. Only normalized Delivered contributes to revenue sums; this is the code's reporting convention, not verified payment-gateway collection.
5. CSV/PDF/print share column definitions and cell normalization. Several report handlers export the full filtered collection rather than only the visible pagination page.
6. A report can only include its loaded records. Check pagination and loading/errors before interpreting a displayed total as a complete database total.

Example to explain

A PO approved today for delivery next week can appear in an approval-dated PO report today, while its delivery workload belongs to next week.

Code basis: src/components/portals/admin/sections/reports/report-data-plan.ts; use-report-datasets.ts; export-utils.ts; ../report-date-utils.ts; src/lib/report-metrics.ts: normalizeOrderReportStatus, isRevenueRecognized

Purchase Requests and Purchase Orders reports

Presentation script

"These two reports describe different documents. The PR report summarizes requested work and decisions. The PO report includes issued purchase orders and their fulfillment stages, not every request that was ever submitted."

How the logic works

1. PR rows are selected by the purchase-request-document helper. Date, status and text filters produce the filtered PR collection.
2. PR cards count total, approved, pending approval, and rejected/cancelled. Requested value sums amounts except rejected and cancelled requests.
3. PO rows exclude RETAIL_POS and require a real issued PO number. A pending PR or replacement delivery does not become a PO solely because it has an order ID.
4. PO report date uses its purchase-order date/approval basis. The report stage and amount come from shared purchase-document helpers.
5. PO cards count total, Delivered, Processing and Cancelled. Purchase value excludes cancelled-stage rows; it is not restricted to delivered revenue.
6. Daily charts and exported rows are derived from the respective filtered document collections. A retained cancelled row remains auditable even when its value is excluded.

Example to explain

Three PRs worth PHP 500 each include one cancelled request. The PR count can remain three, while requested value is PHP 1000. If only one has an issued PO number, the PO list contains that issued document.

Code basis: src/components/portals/admin/sections/reports/purchase-requests-report.tsx; purchase-orders-report.tsx; shared purchase-document helpers

Transaction Records and Retail Sales reports

Presentation script

"Transaction Records combines online and retail records while avoiding duplicate sale entries. Retail Sales focuses on the counter-sales channel and compares recognized totals over its chosen periods."

How the logic works

1. Transaction Records excludes scheduled replacement deliveries and RPL-prefixed orders because they resolve earlier sales rather than create another ordinary sale.
2. Retail channel aliases map to Retail; other order rows map to Wholesale/Online. Separate retail rows are appended only if no matching transaction number or ID exists.
3. Transaction count covers filtered rows; volume sums only recognized revenue rows. Average value divides that recognized volume by the recognized-row count.
4. Paid count checks the payment label for non-cancelled rows. Some absent payment labels default from delivery/completion status, so a Paid label alone is not proof of a gateway event.
5. Retail period metrics calculate recognized sales, recognized transaction count and average. Growth = $(\text{current} - \text{previous}) / \text{previous} \times 100$; if previous is zero, positive current uses 100%, otherwise 0%.
6. Retail metric cards use current/previous period collections; table search/category controls can further filter displayed current rows. Do not assume every card always follows every table filter.

Example to explain

Current recognized retail sales PHP 1500 and previous PHP 1000 produce 50% growth. A cancelled counter sale remains visible in records but does not add recognized sales.

Code basis: `src/components/portals/admin/sections/reports/transactions-report.tsx`; `retail-sales-report.tsx`; `src/lib/report-metrics.ts`

Logistics and Replacement Records reports

Presentation script

"Logistics reports on trips and stop progress. Replacement Records reports on claims and resolution quantities. Neither should be read as a second ordinary sales total."

How the logic works

1. Logistics maps trips to report rows with driver, warehouse, status, date and drop-point counts. Date/status/driver/search filters narrow those rows.
2. Cards count total/completed/in-progress/planned trips, total drop points and completed-drop figures. Some rows use stored completedDropPoints before falling back to COMPLETED stop counts.
3. The execution workflow stores resolved terminal stops in completedDropPoints; therefore a logistics progress figure using that field can include failures/skips/cancellations, not only successful deliveries.
4. Replacement rows combine replacement data with source-order details and normalized resolution status. Filters include the relevant date/status/reason/search controls.
5. Replacement KPIs count resolved/closed, in-progress/follow-up and reported/pending groups. Total units uses the normalized raw bottle quantity when provided, otherwise the row quantity.
6. Replacement export can expand line items so product, quantity, unit and reason remain identifiable instead of collapsing unlike units into an unexplained case total.

Example to explain

A trip with four resolved stops, of which one failed, can show four in a resolved-progress counter. That is different from three successful delivery stops.

Important implementation detail

Legacy normalized report labels RESOLVED/CLOSED are reporting groups, not necessarily the raw ReplacementStatus value stored by a current customer claim.

Code basis: src/components/portals/admin/sections/reports/logistics-report.tsx; replacement-records-report.tsx; backend/core/views_trip_ops.py

Top Clients report and customer ranking

Presentation script

"The Top Clients report aggregates activity by customer. Order count measures the customer's eligible activity, while revenue only counts recognized delivered activity. The top client card is ranked by revenue even if the table is sorted another way."

How the logic works

1. Cancelled and rejected order states are excluded from the underlying eligible order set. The selected period filters by order creation date.
2. Rows are grouped by customer ID, then email, then name fallback. Client records supplement identity and address details.
3. Every eligible order increments order/transaction count. Only `isRevenueRecognized` rows add amount and delivered count.
4. Totals include client count, sum of recognized amounts, and average revenue per client. The #1 card sorts by `totalAmount`, with order count as a tie-breaker.
5. The table may sort by amount, number of orders, or most recent transaction and may be searched by client identity or barangay.
6. New-versus-returning classification uses whole order history rather than only the selected period, so a long-standing buyer is not relabeled new by a short date filter.

Example to explain

Client A has five orders but only two delivered; Client B has three delivered. Their order rankings and revenue rankings can legitimately differ.

Code basis: `src/components/portals/admin/sections/reports/top-clients-report.tsx`; `src/lib/report-metrics.ts`: `summarizeCustomerMix`, `isRevenueRecognized`

Warehouse capacity and inventory reports

Presentation script

"Warehouse reporting explains storage use. Inventory reporting explains stock condition, movements, and product performance. Storage space and product sales should not be treated as the same measure."

How the logic works

1. The Warehouse tab focuses on capacity versus stored quantity and its utilization trend. Utilization percentage is stored quantity divided by capacity times 100 where capacity is positive.
2. Inventory status uses availability and thresholds, with separate Healthy, Low, Critical, Out of Stock and Overstocked calculations described in the stock sections.
3. Movement summaries count records and total IN versus OUT quantities. Reservation events are not automatically counted as physical OUT; unit labels still matter.
4. Inventory report dependencies include inventory, transactions, batches, warehouses, orders and retail sales. Product size/packaging labels are retained so different SKUs can be distinguished.
5. The product-performance view derives daily velocity over the chosen period. All Time uses observed history duration; Today uses one day; custom dates use their date span rather than a fixed monthly divisor.
6. Batch-health and expiry/disposal interpretation use batch dates and retained loss snapshots. A high physical stock count does not prove all that stock remains sellable.

Example to explain

1000 cases of storage with 750 recorded cases means 75% utilization. A product's daily sales velocity is a separate calculation; dividing warehouse utilization by days would not measure it.

Code basis: `src/components/portals/admin/sections/reports/warehouse-report-tab.tsx`; `inventory-report-tab.tsx`; `warehouse-inventory-report.tsx`; `src/lib/report-metrics.ts`

Feedback analytics and satisfaction calculations

Presentation script

"Feedback analytics separates rating value from what the customer actually mentioned. An unrated message counts as feedback, but it does not silently become a zero-star review."

How the logic works

1. Valid rated rows have rating 1-5. Average = sum of those ratings / rated count, rounded to two decimal places.
2. Positive ratings are 4-5, neutral is 3, negative is 1-2. Positive/neutral/negative percentages divide by ratedCount, not all messages.
3. Reason parsing and classification group recognized reasons into report dimensions/issues. A comment must actually mention a dimension to contribute to that dimension's coverage.
4. Report views include rating distribution, comparison deltas, reason/dimension breakdowns, negative issues, an attention queue and participation summaries.
5. The monthly satisfaction-trend panel is explicitly based on all feedback, not only the selected range. Its population can therefore differ from the filtered headline cards.
6. Comparison deltas should be read with meaning: a falling negative rate is an improvement even though its arithmetic difference is negative.

Example to explain

Ratings 5, 4 and 2 plus one unrated message give four feedback records, three rated records, average 3.67, and positive rate $2/3 = 66.67\%$ (subject to displayed rounding).

Code basis: `src/lib/report-metrics.ts: getFeedbackPolarity, summarizeFeedbackKpis, buildFeedbackDimensionBreakdown;`
`src/components/portals/admin/sections/reports/feedback-report-tab.tsx`

Feedback, notifications, settings, and sync

Presentation script

"Feedback gives customers a way to describe their experience. Notifications tell the relevant users about workflow changes. The portals also check change counters so they know when to refresh their own records."

How the logic works

1. Feedback requires a message. If a rating is supplied, it must be an integer from 1 to 5. A customer cannot attach feedback to another customer's order, and duplicate feedback for the same customer and order is rejected.
2. Workflow handlers create customer or staff notifications and invoke the project's email/push transports where configured. Delivery depends on those transports and connectivity.
3. Inventory alerts fire on downward boundaries: positive availability reaching zero, or availability crossing from above the threshold to at-or-below it. Repeated saves while already low do not automatically generate the same alert again.
4. The low-stock alert signal runs after the inventory transaction commits. Its case-availability calculation uses physical and reserved counters, so it is not identical to the UI's early-warning band.
5. The sync endpoint returns revision counters rather than full records. A signed-in portal detects a changed counter and refreshes the relevant collection.
6. Settings update allowed profile and security fields; sensitive account changes use the verification and permission rules described earlier.

Example to explain

With threshold 15, moving from 19 to 18 cases can show the UI warning without triggering the threshold-crossing email. Moving from 16 to 15 can trigger the restock alert; moving from 15 to 14 does not cross that boundary again.

Important implementation detail

Polling and refresh provide cross-device updates. They do not imply that every device changes at exactly the same instant or that every email is guaranteed to arrive.

Code basis: backend/core/views_feedback.py; views_notifications.py; inventory_signals.py; views_sync.py; views_users.py; views_auth.py

In-app notifications, read state, and push devices

Presentation script

"Each account has its own notification feed. Reading a notification updates that account's read state. Push notifications use a registered browser or mobile device and depend on configured delivery services."

How the logic works

1. GET notifications filters by authenticated staff user ID or customer ID, orders newest first, and returns unreadCount. The requested limit is clamped to 1-500.
2. PATCH can mark all unread entries or selected IDs as read and record read_at. DELETE clears the scoped account's notification records.
3. Push GET reports whether web/native transports are configured. Web registration needs endpoint plus encryption keys; native registration uses an Android/iOS token.
4. A device endpoint is transferred to the currently authenticated account during registration, removing a different owner's subscription to that same endpoint.
5. Removing a subscription is owner-scoped. Missing transport configuration returns an unavailable error rather than confirming delivery.
6. In-app creation, successful subscription, provider dispatch and device display are separate steps. Read status also does not prove the user completed the linked business action.

Example to explain

Marking an order notification read changes the unread badge; it does not approve, cancel or deliver that order.

Code basis: backend/core/views_notifications.py: notifications_collection, push_subscriptions_collection; push_notifications.py

Profile changes and security settings

Presentation script

"Profile settings let a user maintain permitted personal details and security preferences. Sensitive identity changes require verification, and operational fields such as staff privileges remain role-controlled."

How the logic works

1. Profile forms load the signed-in account and submit permitted fields to the relevant user/customer/driver endpoint. Avatar uploads return a stored reference.
2. Names and Philippine phone inputs have validation in their relevant update paths. Email uniqueness is checked instead of overwriting another account's identity.
3. Self-service email changes can require verification of both old and new email. Password changes use the OTP/proof flow and the shared password-strength validation.
4. Two-factor and login-alert flags are saved as account settings. Staff session-timeout changes have a minimum of five minutes; customer idle behavior uses the session policy described earlier.
5. Administrator privilege changes and driver service-area changes are not ordinary customer profile fields. Warehouse driver operational profile edits are separate from administrator account management.
6. Some driver notification-preference values are stored client-side. Their presence should not be interpreted as proof that all backend notification categories are filtered by those local preferences.

Example to explain

Changing an avatar updates a profile asset; changing a password changes authentication state and can invalidate older sessions. They are intentionally different save flows.

Code basis: backend/core/views_users.py: user_detail, customer_detail; views_driver.py: driver_profile; views_media.py: upload_customer_avatar; src/components/portals/driver/sections/profile/profile-view.tsx

Synchronization, search, filtering, and data freshness

Presentation script

"The portals use shared change counters and refresh events to keep their own collections current. Search and filters narrow what is shown; they do not change the stored records."

How the logic works

1. The authenticated sync-stamps endpoint returns revision counters and server time, not the underlying orders/inventory. Cache-Control disables stale caching of that response.
2. A changed scope tells a client which collections need reloading. Mutation handlers also emit shared refresh events for affected data such as orders, inventory, customers and auth.
3. Admin global search maps matching entities to the relevant view: requests/orders, clients, trips, warehouses, inventory/products, drivers/vehicles, replacements, feedback and users.
4. Per-view search, status, date, warehouse and pagination controls apply to the collection loaded by that screen. The display must distinguish loading/failure from a legitimate empty result.
5. Customer order/tracking refresh behavior depends on the active view and whether delivery has finished. Tracking need not keep requesting a live route for a delivered order.
6. A cached screen may temporarily show older data. Server validation still controls a mutation when the current stock or status has changed since the list was loaded.

Example to explain

Two staff devices view 20 available cases. One reserves 15; sync leads the other to reload to 5. If the other submits before refresh, server reservation checks are the last authority.

Code basis: backend/core/views_sync.py; sync_stamps.py; src/lib/data-sync.ts; src/components/portals/admin/AdminPortal.tsx; customer/CustomerPortal.tsx

One complete demonstration scenario

Presentation script

"I will demonstrate the system as one connected workflow: receive stock, submit a customer request, approve it, assign delivery, complete the stop, and review the resulting records."

How the logic works

1. Stock-in: begin with 100 physical and sellable cases after an ordinary intake. The threshold is 15. There are no reservations, expired batches, or loose units in this example.
2. Request: a customer requests 20 cases. Reservation makes availability 80 while physical quantity remains 100.
3. Approval: warehouse staff approve the PR. It receives a PO identity; an existing reservation is not counted a second time.
4. Planning: assign the PO to a valid trip. The PO becomes Processing. The driver confirms the load and starts on the scheduled date.
5. Delivery: the order becomes Out for Delivery. The driver completes the stop with a POD photo and applicable empty-container counts. Delivery consumes the 20 cases and settles their reservation.
6. Review: physical stock is now 80, reserved stock is zero, and availability remains 80. The saved threshold stays 15 until an applicable restock recalculates it.

Example to explain

If later sales and requests reduce available stock to 18, the inventory list shows Needs Restocking. At 15 it is critical internally; at zero it is Out of Stock. Staff then review a new stock-in, subject to capacity and overstock checks.

Important implementation detail

These numbers are a teaching example, not a snapshot of the production database. Returned empties and deposits should be demonstrated separately when the example product is returnable.

Code basis: backend/core/views_stock.py, views_orders.py, inventory_allocation.py, views_trips.py, views_trip_ops.py

Quick answers for a presentation or defense

Presentation script

"The central idea is that the system separates requests, reservations, physical stock, delivery results, and container balances. That separation allows the portals to show the right status at each stage."

How the logic works

1. Why does restocking appear before reaching the threshold? The UI has an early-warning band up to 120% of the threshold; critical and low both map to Needs Restocking.
2. Does the threshold fall with every sale? No. Ordinary intake recalculates it; deductions preserve it, and large stock-in can retain the prior threshold for overstock detection.
3. Does placing an order immediately remove physical stock? A fulfillable request reserves stock. Delivery consumes it. Retail completion consumes immediately.
4. Why can an insufficient-stock request still exist? Submission keeps that request for warehouse review, while approval must pass the reservation check.
5. Does trip completion mean every delivery succeeded? No. Stops must be resolved; unsuccessful outcomes remain distinct from delivered orders.
6. Do declared empty bottles automatically prove a return? No. Physical collection verification is separate; unverified declarations can remain pending.
7. Are maps and ETA exact? No. The driver route uses road-routing data, while the inspected customer API also has a simpler distance/speed estimate.

Example to explain

Use the stock example when explaining the difference between 100 physical cases and 80 available cases. The missing 20 are reserved for an order, not necessarily lost, sold at the counter, or already delivered.

Important implementation detail

This document explains inspected implementation behavior. It does not claim a full runtime, security, or deployment audit.

Code basis: Source cross-checks: report-metrics.ts; inventory_overstock.py; inventory_allocation.py; views_orders.py; views_trip_ops.py; deposit_lifecycle.py; views_customer.py

Status families that must not be confused

Presentation script

"A single transaction can have a request decision, an order fulfillment status, a trip status, a stop result, and a deposit-claim status. They describe different records, so the same word can have a different operational meaning."

How the logic works

- 1. Purchase-request approval records the business decision. PO stage records fulfillment; a cancelled fulfillment does not necessarily erase its historical approved request.
- 2. PREPARING is displayed as Processing. A trip IN_PROGRESS is not the same field as replacement IN_PROGRESS.
- 3. Model status enums and normalized report labels are different layers. Legacy aliases can be accepted or grouped by helpers without being mandatory workflow steps.
- 4. Reservation release makes stock available; consumption records fulfillment. Deposit-claim settlement describes returned-container credit, not beverage-stock reservation.

Record	Main states / meaning
Request	PENDING_APPROVAL, APPROVED, REJECTED, CANCELLED
Order	PENDING, APPROVED, PREPARING, RESCHEDULED, OUT_FOR_DELIVERY, DELIVERED, REJECTED, CANCELLED
Trip	PLANNED, IN_PROGRESS, COMPLETED, CANCELLED
Stop	PENDING, IN_TRANSIT, ARRIVED, COMPLETED, FAILED, SKIPPED; execution also uses CANCELLED
Stock reservation	RESERVED, CONSUMED, RELEASED
Retail sale	OPEN, RESERVED, COMPLETED, CANCELLED in model; current completion creates COMPLETED
Deposit claim	PENDING -> SETTLED / PARTIAL / REJECTED; cancellation/release handling is separate

Example to explain

A PO can have approved request history while fulfillment is CANCELLED. Reporting both facts preserves what happened rather than rewriting the original approval.

Code basis: backend/core/models.py; purchase_documents.py; views_orders.py; fleet_sync.py; empties_verification.py; src/lib/report-metrics.ts

Verified implementation limits and discrepancies

Presentation script

"This script describes the reviewed implementation, including places where the frontend, helper rules, and routed backend do not fully align. Describing the limits is part of explaining the complete system accurately."

How the logic works

1. Mixed-case quote and saved-route frontend requests are present, but `/api/mixed-cases/quote` and `/api/trips/saved-routes` are absent from the inspected active Django route list. Their complete operation is not verified.
2. The separate `core/rgb` route module is not included by the inspected config URL entry. Its shared services can still be used by active endpoints; its standalone URLs must not be assumed live.
3. GET `/api/bottle-returns` currently returns an empty returns list. This is not evidence of a functioning generic return-history endpoint.
4. Restock UI warning, stock alert signal, backend overstock flag and dashboard low-stock metrics use distinct conditions. Frontend fallback overstock uses 3 x threshold plus seven days; the supplied backend flag uses the latest-intake/current-stock 10 x rule.
5. Customer cancellation specifically blocks Processing, Delivered and Rejected in its endpoint; it does not explicitly block Out for Delivery. Stock-in empty consumption also differs from older UI wording that says stock-in never deducts empties.
6. Some display defaults, activity times, legacy status aliases and fallback capacity values are not persisted audit facts. No payment gateway, globally optimal routing, live traffic, full offline operation, or complete supplier procurement workflow is established by the reviewed feature paths.

Example to explain

If asked whether the mixed-case quote screen is fully working, the accurate answer is: the frontend and normalization rules exist, but I do not have enough evidence to determine that end-to-end behavior from the current route registry.

Important implementation detail

Review basis: local source, September 27, 2026. This documentation task did not execute every workflow against a live database or change application code.

Code basis: `backend/config/urls.py`; `backend/core/urls.py`; `views_bottles.py`; `inventory_overstock.py`; `inventory_signals.py`; `views_customer.py`; frontend call sites and shared metrics

Active endpoint index 1

Presentation script

"This index lists every route registered in the inspected core API. The owner identifies the source to consult for exact method and permission behavior."

How the logic works

1. Paths are read from backend/core/urls.py and prefixed by /api/ in backend/config/urls.py. Parameter placeholders identify a record ID, not a fixed URL.

Route under /api/	Owning handler
(root)	views_auth.py / api_root
health	views_auth.py / health
health/ready	views_auth.py / health_ready
sync/stamps	views_sync.py / sync_stamps
auth/login	views_auth.py / auth_login
auth/login/verify-otp	views_auth.py / auth_login_verify_otp
auth/unified/login	views_auth.py / auth_unified_login
auth/unified/google	views_auth.py / auth_unified_google
auth/staff/google	views_auth.py / auth_staff_google
auth/customer/login	views_auth.py / auth_customer_login
auth/customer/google	views_auth.py / auth_customer_google
auth/register	views_auth.py / auth_register
auth/email-verification/request	views_auth.py / auth_email_verification_request
auth/email-verification/confirm	views_auth.py / auth_email_verification_confirm
auth/email-verification/request-existing	views_auth.py / auth_email_verification_request_existing
auth/email-verification/confirm-existing	views_auth.py / auth_email_verification_confirm_existing
auth/me	views_auth.py / auth_me
auth/activity	views_auth.py / auth_activity

Example to explain

Health is service status; health/ready additionally checks readiness. Neither is a customer purchase or inventory action.

Code basis: backend/config/urls.py; backend/core/urls.py

Active endpoint index 2

Presentation script

"This index lists every route registered in the inspected core API. The owner identifies the source to consult for exact method and permission behavior."

How the logic works

1. Paths are read from backend/core/urls.py and prefixed by /api/ in backend/config/urls.py. Parameter placeholders identify a record ID, not a fixed URL.

Route under /api/	Owning handler
auth/logout	views_auth.py / auth_logout
auth/password-reset/request-otp	views_auth.py / auth_password_reset_request_otp
auth/password-reset/verify-otp	views_auth.py / auth_password_reset_verify_otp
auth/password-reset/reset	views_auth.py / auth_password_reset_reset
roles	views_users.py / roles_list
users	views_users.py / users_collection
users/<str:user_id>	views_users.py / user_detail
customers	views_users.py / customers_collection
customers/<str:customer_id>	views_users.py / customer_detail
categories	views_users.py / categories_list
warehouses	views_warehouses.py / warehouses_collection
warehouses/<str:warehouse_id>	views_warehouses.py / warehouse_detail
products	views_products.py / products_collection
products/<str:product_id>	views_products.py / product_detail
inventory	views_inventory.py / inventory_collection
inventory/empty-cases	views_inventory.py / empty_case_inventory
inventory/empty-returns	views_inventory.py / record_returned_empty_containers
inventory/<str:inventory_id>	views_inventory.py / inventory_detail

Example to explain

Health is service status; health/ready additionally checks readiness. Neither is a customer purchase or inventory action.

Code basis: backend/config/urls.py; backend/core/urls.py

Active endpoint index 3

Presentation script

"This index lists every route registered in the inspected core API. The owner identifies the source to consult for exact method and permission behavior."

How the logic works

1. Paths are read from backend/core/urls.py and prefixed by /api/ in backend/config/urls.py. Parameter placeholders identify a record ID, not a fixed URL.

Route under /api/	Owning handler
inventory-transactions	views_inventory.py / inventory_transactions_list
stock-batches	views_stock.py / stock_batches_collection
stock-batches/bulk	views_stock.py / stock_batches_bulk_collection
stock-batches/expired-stock	views_inventory.py / resolve_expired_stock
stock-batches/disposals	views_inventory.py / disposed_stock_history
vehicles	views_fleet.py / vehicles_collection
vehicles/<str:vehicle_id>	views_fleet.py / vehicle_detail
drivers	views_fleet.py / drivers_collection
dashboard/stats	views_dashboard.py / dashboard_stats
feedback	views_feedback.py / feedback_collection
notifications	views_notifications.py / notifications_collection
push-subscriptions	views_notifications.py / push_subscriptions_collection
retail/products	views_retail.py / retail_products
retail/quote	views_retail.py / retail_quote
retail/sales	views_retail.py / retail_sales_collection
retail/sales/<str:sale_id>	views_retail.py / retail_sale_detail
retail/sales/<str:sale_id>/cancel	views_retail.py / retail_sale_cancel
orders	views_orders.py / orders_collection

Example to explain

Health is service status; health/ready additionally checks readiness. Neither is a customer purchase or inventory action.

Code basis: backend/config/urls.py; backend/core/urls.py

Active endpoint index 4

Presentation script

"This index lists every route registered in the inspected core API. The owner identifies the source to consult for exact method and permission behavior."

How the logic works

1. Paths are read from backend/core/urls.py and prefixed by /api/ in backend/config/urls.py. Parameter placeholders identify a record ID, not a fixed URL.

Route under /api/	Owning handler
replacements	views_customer.py / replacements_collection
orders/<str:order_id>	views_orders.py / order_detail
orders/<str:order_id>/status	views_orders.py / order_status_update
trips	views_trips.py / trips_collection
driver/trips	views_driver.py / driver_trips
uploads/product-image	views_media.py / upload_product_image
uploads/pod-image	views_media.py / upload_pod_image
uploads/damage-image	views_media.py / upload_damage_image
uploads/customer-avatar	views_media.py / upload_customer_avatar
uploads/replacement-evidence	views_media.py / upload_replacement_evidence
media/<path:path>	views_media.py / private_media
customer/orders	views_customer.py / customer_orders
customer/orders/<str:order_id>/deposit-refund	views_customer.py / customer_order_deposit_refund
customer/orders/<str:order_id>/cancel	views_customer.py / customer_order_cancel
customer/replacements	views_customer.py / customer_replacements
customer/replacements/<str:replacement_id>/cancel	views_customer.py / customer_replacement_cancel
customer/tracking	views_customer.py / customer_tracking
customer/empty-bottles/eligible	views_bottles.py / customer_empty_bottles_eligible

Example to explain

Health is service status; health/ready additionally checks readiness. Neither is a customer purchase or inventory action.

Code basis: backend/config/urls.py; backend/core/urls.py

Active endpoint index 5

Presentation script

"This index lists every route registered in the inspected core API. The owner identifies the source to consult for exact method and permission behavior."

How the logic works

1. Paths are read from backend/core/urls.py and prefixed by /api/ in backend/config/urls.py. Parameter placeholders identify a record ID, not a fixed URL.

Route under /api/	Owning handler
customer/empty-bottles/record	views_bottles.py / customer_record_empty_bottles
bottle-returns	views_bottles.py / bottle_returns_collection
driver/location	views_driver.py / driver_location
driver/profile	views_driver.py / driver_profile
trips/route-plan	views_trip_ops.py / trips_route_plan
trips/upcoming-deliveries	views_trip_planning.py / trips_upcoming_deliveries
trips/<str:trip_id>	views_trips.py / trip_detail
trips/<str:trip_id>/start	views_trip_ops.py / trip_start
trips/<str:trip_id>/complete	views_trip_ops.py / trip_complete
trips/<str:trip_id>/drop-points/<str:drop_point_id>	views_trip_ops.py / trip_drop_point_update
trips/<str:trip_id>/stops/<str:stop_id>	views_trip_ops.py / trip_stop_update
trips/<str:trip_id>/unassign	views_trips.py / trip_unassign_items
trips/check/<str:trip_number>	views_trips.py / trip_check

Example to explain

Health is service status; health/ready additionally checks readiness. Neither is a customer purchase or inventory action.

Code basis: backend/config/urls.py; backend/core/urls.py